RACE F1 INTELLIGENCE PLATFORM

RACE F1 Simulator & AI Strategy Engine

Complete Technical Architecture, Replay Core, Analytical Models & Provider-Independent AI System

SYSTEM VERSION
RACE Engine v2.4 (Checkpoints 1–5C + NIM Integration)

ACTIVE AI PROVIDERS
NVIDIA NIM (LLaMA 3.2 11B) & Google Gemini (2.5/3.6 Flash)

SUPPORTED SESSIONS
2022 Monza Italian GP & 2023 Monza Italian GP

TEST VERIFICATION
47 Passed (30 API Unit/Mock + 7 UI E2E + 10 REST Core)

BACKEND TECHNOLOGY
FastAPI, Python 3.10, FastF1, NumPy, Scipy, Scikit-learn

FRONTEND TECHNOLOGY
React 18, TypeScript, Vite, Tailwind CSS, Playwright

1. Executive Summary & System Overview

RACE (Race Intelligence & Strategy Simulator) is an enterprise-grade Formula 1 replay simulation, predictive analytics, and interactive strategy platform. It combines sub-second physical replay telemetry with deterministic mathematical modeling and a bounded, strictly grounded AI Strategy Assistant capable of operating seamlessly over multiple enterprise AI providers (**NVIDIA NIM** and **Google Gemini**).

🏎️ Authoritative Telemetry Core

Continuous spatial interpolation (20 transponders, X, Y, Z trajectory coordinates, speed, throttle, brake, gear, RPM, DRS status) anchored to strictly bounded point-in-time session timestamps (`T_0` to `T_finish`).

📊 Deterministic Analytical Engines

Authoritative micro-sector lap comparisons, compound-specific tyre degradation curves, pit loss waterfall simulations, and statistical overtake probability models calculated with zero hallucination.

🤖 Provider-Independent AI Assistant

Strategy orchestrator operating across NVIDIA NIM (OpenAI-compatible) and Google Gemini (GenAI SDK), featuring bounded tool turns (max 3), strict evidence grounding, and automatic API key detection.

⚡ Interactive Operations UI

React 18 + Vite interface delivering synchronized track outlines, live speed traces, interactive driver comparisons, pit waterfall diagrams, and an accessible AI assistant drawer with deep-linking evidence chips.

High-Level System Architecture

The system strictly isolates authoritative physical telemetry calculations from language model generation. The LLM acts solely as a natural-language query parser and synthesizer, while all numbers, gaps, lap times, degradation rates, and predictions are supplied deterministically by the RACE engine.


RACE F1 WEB CLIENT (React 18 + Vite)
[Circuit Map] [Telemetry Canvas] [Leaderboard Scrubber] [Strategy Tabs] [AI Assistant Drawer]
HTTP REST / WebSocket Streaming
FASTAPI APPLICATION LAYER (Backend Core)
- /api/races (Session Catalog)
- /api/races/{race_id}/track (Geometry)
- /api/races/{race_id}/assistant/query
- /api/races/{race_id}/state (Replay Point-in-Time)
- /api/races/{race_id}/assistant/status
- /ws/races/{race_id} (Live Clock Synchronization)
SESSION & TELEMETRY CORE
- FastF1 Cached Data Store
- Spline Spatial Replay
- Transponder Sync Engine
- Point-in-Time Clocks
BOUNDED AI STRATEGY ORCHESTRATOR
- Turn Loop Manager (Max 3 turns, cycle deduplication)
- Unmeasured Domain Guardrails (Driver psychology/chassis reject)
- Evidence Grounding & Chip Reference Synthesizer
DETERMINISTIC ENGINES
1. LapComparisonEngine
2. TyreDegradationEngine
3. PitStrategyEngine
4. BattlePredictionEngine
TOOL DISPATCH REGISTRY
1. get_race_leaderboard
2. compare_driver_laps
3. get_tyre_stint_degradation
4. get_pit_strategy_and_waterfall
5. get_battle_and_overtake_prediction
COMMON LLM PROVIDER LAYER
(LLMProvider Contract)
NvidiaLLMProvider
- OpenAI-Compatible Endpoint
- meta/llama-3.2-11b-vision-instruct
- Function Tools & Tool-Call IDs
- Cooldown & Key Redaction
GeminiLLMProvider
- Google GenAI SDK Client
- gemini-2.5-flash / gemini-3.6-flash
- Schema Function Declarations
- Quota Timing Extraction

2. Replay Engine & Deterministic Analytics

The RACE computational backend guarantees total mathematical repeatability and point-in-time state preservation. Race state is never mutated by inspection queries; querying `as_of=4500.0s` returns the exact state of all 20 cars at session time 4500.0s without altering the active replay clock.

2.1 Authoritative Session Clocks & Clamping

- `T_0`: Formation lap departure / start of authoritative timing transponders (e.g., Monza 2022: 3748.384s).
- `T_finish`: Final checkered flag crossing for the final classified runner.
- **Boundary Clamping**: Any request with `as_of < T_0` is automatically clamped to `T_0`; any request with `as_of > T_finish` is clamped to `T_finish`.

2.2 Deterministic Strategy Engines

ENGINE	KEY CALCULATIONS	INPUTS & SCOPE	AUTHORITATIVE GUARANTEES
LapComparisonEngine	Micro-sector delta analysis, corner minimum speeds, apex acceleration points, mean delta: `mean_delta_s = lap_time_a - lap_time_b`.	Driver A, Driver B, Lap numbers, race session.	Positive delta strictly indicates Driver A is slower than Driver B. Telemetry traces align on distance meters, not elapsed seconds.
TyreDegradationEngine	Lap-by-lap tyre degradation coefficients, compound wear curves (Soft, Medium, Hard), thermal degradation penalties.	Stint length, compound, age, track temperature.	Linear and exponential wear curves calibrated against clean-air laps. Excludes safety car and yellow flag distortions.
PitStrategyEngine	Pit loss delta, undercut/overcut time windows, pit exit gap predictions, stationary loss assumptions.	Target driver, pit lap, target compound, current gaps.	Stationary time (2.5s) is explicitly labeled an illustrative assumption, not a measured sensor value. Never misrepresented as real-time sensor measurement.
BattlePredictionEngine	DRS zone detection, closing rates, overtake probability curve: `P(overtake) = f(gap, delta_v, tyre_diff, laps_remaining)`.	Leading driver, chasing driver, current delta.	Strictly calibrated as statistical probability models; never reported as guarantees or certainties.

> **Critical Grounding Constraint:** The AI assistant is forbidden from inventing tyre degradation rates, calculating gaps, or claiming pit stop durations independently. All numeric assertions in strategic summaries must be derived directly from deterministic tool outputs.

3. Multi-Provider AI Architecture

To eliminate single-vendor dependency and ensure uninterrupted operations during API quota limits, RACE incorporates a pluggable, provider-independent LLM architecture. Switching between **NVIDIA NIM** and **Google Gemini** requires zero code modifications.

3.1 LLMProvider Abstraction Contract

All providers inherit from `LLMProvider` (defined in `src/race/ai/provider.py`), establishing a normalized interface for synchronous turn generation, asynchronous streaming, tool call extraction, and rate limit cooldowns.

```
class LLMProvider(ABC):
    @abstractmethod
    def generate_turn(
        self,
        messages: List[Dict[str, Any]],
        tools: List[Any],
        system_instruction: str = "",
        temperature: float = 0.2,
    ) → LLMResponse:
        '''Executes a single conversational/tool calling turn.'''
        pass

    @abstractmethod
    def is_available(self) → bool:
        '''Returns True if the provider has valid credentials and is ready.'''
        pass

    @abstractmethod
    def get_provider_type(self) → str:
        '''Returns "nvidia", "gemini", or "mock".'''
        pass

    @classmethod
    def get_quota_cooldown_remaining(cls) → Optional[Tuple[float, str]]:
        '''Returns (remaining_seconds, formatted_string) if in rate-limit cooldown.'''
        pass
```

3.2 Comparison of Provider Implementations

FEATURE / DIMENSION	NVIDIA NIM PROVIDER (`NVIDIALLMPROVIDER`)	GOOGLE GEMINI PROVIDER (`GEMINILLMPROVIDER`)
Primary Model	`meta/llama-3.2-11b-vision-instruct` (Verified Active)	`gemini-2.5-flash` / `gemini-3.6-flash`
Protocol / API	OpenAI-Compatible REST API (`/v1/chat/completions`)	Google GenAI Client SDK (`google-genai`)
Base URL	`https://integrate.api.nvidia.com/v1`	Google Cloud AI Infrastructure
Authentication	`NVIDIA_API_KEY` (Bearer Token, `nvapi-...`)	`GEMINI_API_KEY` (API Key, `AIza...`)
Tool Declaration Spec	Normalized OpenAI function definitions: `{"type": "function", ...}`	Gemini SDK `types.Tool(function_declarations=[...])`
Multi-Turn ID Tracking	Strict tool-call IDs (e.g. `chatcmpl-tool-3b54...`) matched to tool responses	Gemini function call/response parts mapped via function name
Typical Turn-1 Latency	**0.64s – 0.89s** (High-speed NIM inference)	**1.10s – 1.80s**
Cooldown Handling	HTTP 429 extraction with retry header or 60s default cooldown	HTTP 429 quota metric parsing with exact seconds backoff

3.3 Configuration & Automatic Key-Prefix Detection

The system supports explicit provider selection via `AI_PROVIDER`, with seamless auto-detection when unset:

- If `AI_PROVIDER=nvidia` → NVIDIA NIM is activated.
- If `AI_PROVIDER=gemini` → Google Gemini is activated.
- If `AI_PROVIDER=mock` → Offline deterministic mock provider is activated.
- **Automatic Detection**: If `AI_PROVIDER` is omitted, keys starting with `nvapi-` automatically select NVIDIA; keys starting with `AIza` select Gemini.

4. Bounded Orchestrator & Evidence Grounding

The `AIStrategyOrchestrator` guarantees safety, reliability, and precision through five architectural pillars:

4.1 Bounded Tool Calling Loop

- **Maximum 3 Turns**: The orchestrator halts execution after 3 iterations to prevent infinite loops or quota exhaustion.
- **Recursion & Fingerprint Deduplication**: Identical calls (same tool + same arguments) within a single query are caught via fingerprinting: `call_fingerprint = f"{func_name}:{json.dumps(args, sort_keys=True)}"`. Rather than failing, a helpful notice instructs the model to synthesize from existing turn data.
- **Non-blocking Execution**: Network turns run asynchronously via `asyncio.to_thread`, preserving responsiveness across concurrent HTTP sessions.

4.2 Deterministic Tool Registry

The orchestrator exposes five authoritative tools to the model:

TOOL IDENTIFIER	FUNCTION PURPOSE	RETURNED EVIDENCE SCHEMA
`get_race_leaderboard`	Retrieves classified positions, gaps, tyre compounds, ages, and pit counts.	Leaderboard array of 20 drivers, current lap, weather, track status.
`compare_driver_laps`	Calculates lap time delta, micro-sector performance, and corner speeds.	`mean_delta_s`, `fastest_lap_a`, `fastest_lap_b`, sector breakdown.
`get_tyre_stint_degradation`	Calculates tyre wear rate in seconds/lap and estimated remaining life.	`degradation_s_per_lap`, `stint_length`, `compound`, wear percentage.
`get_pit_strategy_and_waterfall`	Computes pit exit position, gap, and time lost in pit lane.	`pit_loss_s`, `projected_exit_position`, `undercut_potential`.
`get_battle_and_overtake_prediction`	Forecasts overtake probability within 3–5 laps between adjacent cars.	`overtake_probability`, `closing_rate_s_per_lap`, DRS effectiveness.

4.3 Strict Evidence Grounding & Guardrails

> **Unmeasured Domain Rejection:** If a user asks about driver emotions, psychological state, engine power modes, or internal chassis damage, the orchestrator immediately intercepts the query with status `CANNOT_TELL` and explains that track timing transponders cannot measure internal or subjective states.

For all valid queries, the orchestrator parses the synthesized answer against raw tool output records:

- **Key Numbers Extraction**: Identifies verified positions, gaps (e.g. `+1.792s`), and lap counts.
- **Evidence Reference Chips**: Generates typed `EvidenceReference` objects containing unique IDs, labels, descriptions, and `nav_payload` coordinates for UI deep-linking.
- **Confidence Rating**: Reports `HIGH`, `MEDIUM`, `LOW`, or `CANNOT_TELL` based on the ratio of grounded claims to total assertions.

5. API Specification & Integration Contracts

5.1 Assistant Status Endpoint: `GET /api/races/{race_id}/assistant/status`

Returns real-time availability, the active provider, model name, and registered tools:


```
{
  "available": true,
  "provider": "nvidia",
  "model": "meta/llama-3.2-11b-vision-instruct",
  "message": "AI Strategy Assistant is ready to accept queries.",
  "tools_registered": [
    "get_race_leaderboard",
    "compare_driver_laps",
    "get_tyre_stint_degradation",
    "get_pit_strategy_and_waterfall",
    "get_battle_and_overtake_prediction"
  ],
  "status_reason": "available"
}
```


5.2 Assistant Query Endpoint: `POST /api/races/{race_id}/assistant/query`

Accepts a natural language query with an optional point-in-time timestamp (`as_of`):


```
// Request Payload
{
  "prompt": "Who is currently leading the race?",
  "as_of": 4500.0,
  "conversation_history": []
}

// Live Verified Response Payload
{
  "status": "SUCCESS",
  "answer": "The current race leader is LEC.",
  "claims": ["The current race leader is LEC."],
  "key_numbers": [
    {"label": "LEC Position", "value": "P1"},
    {"label": "LEC Gap to Leader", "value": "0.000s"},
    {"label": "VER Position", "value": "P2"},
    {"label": "VER Gap to Leader", "value": "1.792s"}
  ],
  "evidence": [{
    "id": "ev-leaderboard-4500",
    "type": "replay",
    "label": "Race Leaderboard (t=4500.0s)",
    "description": "Leaderboard state covering 20 drivers at t=4500.0s.",
    "driver_codes": ["LEC", "VER", "RUS", "RIC", "GAS"],
    "as_of": 4500.0,
    "nav_payload": {"targetTab": "dashboard", "as_of": 4500.0}
  }],
  "confidence": "HIGH",
  "tool_calls_executed": ["get_race_leaderboard"],
  "as_of": 4500.0
}
```

6. Test Verification & Operations Guide

6.1 Comprehensive Test Verification Matrix

Every module in the RACE platform undergoes automated testing prior to deployment:

TEST SUITE	TESTS	DURATION	STATUS	COVERAGE SCOPE
Backend AI API Suite `tests/test_ai_assistant_api.py`	30	4.11s	100% PASS	Provider selection, NVIDIA request/response formatting, multi-tool parsing, auth failure, quota backoff, multi-turn tool execution, status contracts.
UI E2E Playwright Suite `tests/test_ai_assistant_ui_e2e.py`	7	133.11s	100% PASS	Assistant drawer open/close, accessibility landmarks, query execution, loading skeleton, evidence chip navigation, key banner alerts, quota cooldown.
REST Regression Suite `tests/test_api_rest.py`	10	2.61s	100% PASS	Race catalog discovery, state serialization, transponder timing, 404/422 validation, track outline geometry, frontend asset serving.
Frontend Production Build `ui (tsc && vite build)`	1,534 modules	2.09s	SUCCESS	TypeScript strict compilation, zero lint warnings, bundle generation (`dist/assets/index-BwsnoowW.js`).
Live NVIDIA API Verification `Monza 2022 Query (t=4500s)`	Live Query	1.85s	SUCCESS	Deterministic tool invocation (`get_race_leaderboard`), P1 extraction (LEC), HIGH confidence grounding, evidence chip synthesis.

6.2 Environment Variables Reference

Configure these settings in the project root `.env` file:

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
`AI_PROVIDER`	No	*auto-detect by key*	Active AI provider: `nvidia` , `gemini` , or `mock` .
`NVIDIA_API_KEY`	If NVIDIA	—	NVIDIA NIM API key (starts with `nvapi-...`).
`NVIDIA_MODEL`	No	`meta/llama-3.2-11b-vision-instruct`	NVIDIA NIM hosted model name.
`NVIDIA_BASE_URL`	No	`https://integrate.api.nvidia.com/v1`	NVIDIA NIM endpoint URL.
`GEMINI_API_KEY`	If Gemini	—	Google Gemini API key (starts with `AIza...`).
`GEMINI_MODEL`	No	`gemini-2.5-flash`	Google Gemini model name.
`AI_REQUEST_TIMEOUT_S`	No	`35.0`	Timeout per LLM network turn in seconds.

6.3 System Startup Commands

```
# 1. Start the FastAPI Application Server
python -m uvicorn src.race.api.app:app --host 127.0.0.1 --port 8000

# 2. Check Assistant Status over HTTP
curl http://127.0.0.1:8000/api/races/2022_monza/assistant/status

# 3. Run Backend Test Suite
pytest tests/test_ai_assistant_api.py -v

# 4. Build Frontend Bundle
cd ui && npm.cmd run build
```

6.4 Operational Troubleshooting

HTTP 410 Gone on NVIDIA Models: Older LLaMA models (such as `meta/llama-3.1-8b-instruct`, `meta/llama-3.2-3b-instruct`, and `meta/llama-3.2-1b-instruct`) have reached End-of-Life on NVIDIA's cloud catalog. Ensure you configure `meta/llama-3.2-11b-vision-instruct`.

HTTP 429 Rate Limit Cooldown: When an API quota is exhausted, RACE automatically extracts the retry delay and enters a global cooldown. The UI displays an amber alert with the countdown. Replay and deterministic telemetry remain 100% operational.

Security Guarantee: API keys are strictly confined to backend environment memory. Keys are never transmitted to the browser, logged in application traces, or included in error responses.